

32. ANÁLISES ESTRUCTURADA. ANÁLISES ORIENTADA A OBJETOS. LENGUAJE UNIFICADO DE MODELADO (UML). DISEÑO. DISEÑO ESTRUCTURADO. DISEÑO DE DATOS. DISEÑO DEL INTERFACE DE USUARIO. DISEÑO PROCEDIMENTAL. DISEÑO ORIENTADO A OBJETOS.

Tema 32: Análisis estructurado. Análisis orientado a objetos. Lenguaje unificado de modelado (UML). Diseño. Diseño estructurado. Diseño de datos. Diseño de la interfaz de usuario. Diseño procedimental. Diseño orientado a objetos.

32.1 Análisis del sistema de información.

32.1.1 Análisis estructurado.

32.1.2 Análisis orientado a objetos.

32.1.3 Lenguaje unificado de modelado.

32.2 Diseño de un sistema de información.

32.2.1 Diseño estructurado.

32.2.2 Diseño de datos.

32.2.3 Diseño de la interfaz de usuario.

32.2.4 Diseño procedimental.

32.2.5 Diseño orientado a objetos.

32.1 Análisis del sistema de información.

La ingeniería del software empieza con una serie de tareas de modelado que llevan a una especificación completa de los requisitos y a una representación del diseño general del software a construir. El *modelo de análisis*, realmente un conjunto de modelos, es la primera representación técnica de un sistema. Con los años se han propuesto muchos métodos para el modelado del análisis, pero el panorama actual está dominado por dos tendencias: el **análisis estructurado**, método de modelado clásico, cada vez más en desuso, y el **análisis orientado a objetos**, que es la tendencia que domina hoy en día.

32.1.1 Análisis estructurado.

El término análisis estructurado originalmente acuñado por Douglas Ross, fue popularizado por Tom DeMarco en su libro “Análisis Estructurado”, que

fue el primero en establecer los principios por los cuales debe guiarse un proceso de análisis de software. Se basa principalmente en el análisis de los flujos de información de un sistema, permitiendo al analista conocer un sistema o proceso de una forma lógica y manejable, al mismo tiempo que proporciona la base para asegurar que no se omite ningún detalle pertinente.

Según Pressman, el modelo de análisis debe lograr tres objetivos primarios:

1. Describir lo que requiere el cliente.
2. Establecer una base para la creación de un diseño de software
3. Definir un conjunto de requisitos que se pueda validar una vez que se construye el software.

Para lograr estos objetivos, el modelo de análisis extraído durante el análisis estructurado toma la forma ilustrada en la siguiente figura:



En el centro del modelo se encuentra el **diccionario de datos** (un almacén que contiene definiciones de todos los objetos de datos consumidos y producidos por el software). Tres diagramas diferentes rodean el núcleo.

- El **diagrama de entidad-relación** (DER) representa las relaciones entre los objetos de datos. El DER es la notación que se usa para

realizar la actividad de modelado de datos. Los atributos de cada objeto de datos señalados en el DER se puede describir mediante una descripción de objetos de datos.

- El **diagrama de flujo de datos** (DFD) sirve para dos propósitos:
 - a) Proporcionar una indicación de cómo se transforman los datos a medida que se avanza en el sistema
 - b) representar las funciones (y subfunciones) que transforman el flujo de datos.

El DFD proporciona información adicional que se usa durante el análisis del dominio de información y sirve como base para el modelado de función. En una especificación de proceso (EP) se encuentra una descripción de cada función presentada en el DFD.

- El **diagrama de transición de estados** (DTE) indica cómo se comporta el sistema como consecuencia de sucesos externos. Para lograr esto, el DTE representa los diferentes modos de comportamiento (llamados estados) del sistema y la manera en que se hacen las transiciones de estado a estado. El DTE sirve como la base del modelado de comportamiento. Dentro de la especificación de control (EC) se encuentra más información sobre los aspectos de control del software.

Estos tres diagramas permiten modelizar completamente el sistema desde tres puntos de vista:

- **Punto de Vista Funcional.** Se describe cómo se transforman los datos dentro del sistema. Para describir este modelo funcional se crea un modelo de procesos, el cual se construye usando la técnica de los Diagramas de Flujo de Datos.
- **Punto de Vista Estático.** Describe los objetos de información del sistema y cómo se relacionan estos objetos entre ellos. Para crear este modelo estático se construye el modelo de datos, el cual se puede construir usando la técnica del diagrama Entidad – Relación.

- **Punto de Vista Dinámico.** Describe los aspectos del sistema que cambian a lo largo del tiempo. Se describen los estados por los que va pasando el sistema y las condiciones y eventos que hacen que el sistema pase de un estado a otro. Una posible técnica para construir el modelo dinámico es la del Diagrama de Transición de Estados.

La metodología de **Análisis Estructurado Moderno** es una propuesta de Edward Yourdon que establece un nuevo modelo para el análisis estructurado e incorpora las ideas, criterios y herramientas que lo precedieron. Se basa en la construcción del **modelo esencial** del sistema, que indica lo que el sistema debe hacer para satisfacer los requisitos del usuario y debe mencionar lo mínimo posible como el sistema lo llevará a cabo. Está compuesto por dos componentes principales:

- **El Modelo Ambiental:** define la frontera entre el sistema y el resto del mundo (es decir, el ambiente donde el sistema reside). El modelo ambiental está compuesto de un Diagrama de Contexto, una Lista de Eventos y una descripción pequeña del propósito del sistema, la Declaración de Objetivos.
- **El Modelo de Comportamiento:** describe la conducta, del interior del sistema, necesaria para interactuar exitosamente con el ambiente. Esta compuesto de Diagramas de Flujo de Datos, Lista de Eventos, Diagramas de Entidades y Relaciones, Diagramas de Transición de Estados, Diccionario de Datos y Especificación de Procesos.

31.1.2 Análisis orientado a objetos.

Los **objetos** son entidades que tienen atributos (datos) y formas de comportamiento (procedimientos o métodos) particulares. Las **clases** describen un conjunto de objetos diferentes con propiedades (atributos) similares y un comportamiento común. Las clases son un concepto estático definido en el programa fuente, son una abstracción de la esencia de un

objeto, mientras que los objetos son entes dinámicos que existen en tiempo y espacio y que ocupan memoria en la ejecución de un programa. Los objetos interaccionan entre si mediante **mensajes**, que no es más que una solicitud que hace un objeto a otro pidiéndole que se comporte de alguna forma determinada.

Los principios del modelo orientado a objetos son:

- **Identidad:** Cada objeto tiene su propia identidad inherente, es decir, dos objetos son distintos aunque tengan todas sus propiedades iguales.
- **Clasificación.** Se refiere a que los objetos que tienen la misma estructura de datos (atributos), y el mismo comportamiento (operaciones), están agrupados en una clase.
- **Herencia.** La herencia es el mecanismo mediante el cual una clase (subclase) adquiere las propiedades de otra clase jerárquicamente superior (superclase, clase base). La herencia proporciona el mecanismo para compartir automáticamente métodos y datos entre clases y subclases.
- **Abstracción:** Se trata de abstraer los datos y métodos comunes a un conjunto de objetos para almacenarlos en una clase.
- **Encapsulación:** Es el término que se utiliza para expresar que los datos de un objeto sólo pueden ser manipulados mediante los mensajes y métodos predefinidos.
- **Polimorfismo:** Es la propiedad por la cual un mismo mensaje puede originar conductas diferentes al ser recibido por objetos diferentes. También hablamos de polimorfismo cuando tenemos distintos métodos que tienen un comportamiento distinto en función del número o tipo de parámetros que reciben.
- **Reusabilidad:** Es la capacidad de producir componentes reutilizables para otros diseños o aplicaciones.

- **Persistencia:** La persistencia es la cualidad que se refiere a la permanencia del objeto, es decir, al tiempo durante el cual se le asigna espacio y permanece accesible en la memoria del ordenador (principal o secundaria).
- **Extensibilidad:** Es la capacidad de un programa para ser fácilmente alterado de forma que pueda tratar con nuevas clases de entrada.

El Análisis Orientado a Objetos es un método de análisis que examina los requisitos desde la perspectiva de las clases y objetos encontrados en el vocabulario de del dominio del problema (Booch). Debe ser comprendido por el cliente y servir de ayuda para encontrar los verdaderos requerimientos del sistema. Firesmith describe el **análisis del dominio** como “la identificación, análisis y especificación de requisitos comunes en un dominio de aplicación específico, normalmente para su reutilización en múltiples proyectos dentro del mismo dominio de aplicación. El análisis orientado a objetos del dominio es la identificación, análisis y especificación de capacidades comunes y reutilizables dentro de un dominio de aplicación específico, en términos de objetos, clases, submontajes y marcos de trabajo comunes”. Al igual que en los métodos estructurados tradicionales, en la etapa Análisis hay que establecer qué es lo que debe hacerse, dejando para etapas posteriores los detalles. El resultado del análisis debe ser una completa comprensión del problema. Las dos grandes etapas de que consta el Análisis son las siguientes:

1. La descripción o especificación del problema. Esta descripción no debe considerarse inmutable, sino más bien como la base para ir refinando las especificaciones reales. La especificación del problema debe comprender el establecer el ámbito del problema, describir las necesidades y requisitos, el contexto de la aplicación, los supuestos de que se parte o las necesidades de rendimiento del sistema. En estas especificaciones, el usuario del sistema debe indicar cuáles son obligadas y cuáles se pueden considerar opcionales. Asimismo, otros puntos a tratar pueden ser los estándares de Ingeniería del Software,

diseño de las pruebas a efectuar, previsión de futuras extensiones, etc.

2. La modelización del Análisis: Las características esenciales deben abstraerse en un modelo. Las especificaciones expresadas en lenguaje natural tienden a ser ambiguas, incompletas e inconsistentes, sin embargo, el Modelo de Análisis es una representación precisa y concisa del problema, que permite construir una solución. La etapa siguiente de diseño se remitirá a este modelo, en lugar de a las vagas especificaciones iniciales. El Modelo de Análisis se construye identificando las clases y objetos del dominio del problema (estructura estática), las interacciones entre los objetos y su secuenciamiento (estructura dinámica), y las acciones a realizar por el sistema que producen un resultado observable y valioso para los usuarios (estructura funcional).

32.1.3 Lenguaje unificado de modelado (UML).

UML (Unified Modeling Language) es un lenguaje que permite modelar, construir y documentar los elementos que forman un sistema software orientado a objetos. Se ha convertido en el estándar de facto de la industria, debido a que ha sido impulsado por los autores de los tres métodos más usados de orientación a objetos: Grady Booch, Ivar Jacobson y Jim Rumbaugh. Es el estándar actual del llamado Object Management Group (OMG).

UML sirve para *especificar*, modelos concretos, no ambiguos y completos. Un modelo de UML representa a un sistema software desde una perspectiva específica. Cada modelo nos permite fijarnos en un aspecto distinto del sistema. Debido a su estandarización, y aunque no sea un lenguaje de programación, UML se puede conectar de manera directa a lenguajes de programación como Java, C++ o Visual Basic.

UML se expresa a través de *elementos de construcción*, de *relaciones* y de *diagramas* que contienen elementos y relaciones

Los **elementos de construcción** son los bloques básicos de construcción que serán de cuatro tipos:

1. **Elementos estructurales.** Son las partes estáticas del modelo. Son siete:
 - a) **Clase:** Descripción de un conjunto de objetos que comparten los mismos atributos, operaciones, relaciones y semántica.
 - b) **Interfaz:** colección de operaciones que especifican un servicio de una clase o componente, mostrando el comportamiento visible externamente de ese elemento.
 - c) **Colaboración:** define una interacción y representa un conjunto de elementos del modelo que colaboran para proporcionar un comportamiento cooperativo mayor que la suma de los comportamientos de sus elementos.
 - d) **Caso de uso:** descripción de un conjunto de secuencias de acciones que un sistema ejecuta y que produce un resultado observable de interés para un usuario particular.
 - e) **Clase activa:** Clase cuyos objetos tienen uno o más procesos o hilos de ejecución, y por lo tanto pueden dar origen a actividades de control.
 - f) **Componente:** Parte física y reemplazable de un sistema que representa típicamente el empaquetamiento físico de diferentes elementos lógicos, como clases, interfaces y colaboraciones.
 - g) **Nodo:** Elemento físico que existe en tiempo de ejecución y representa un recurso computacional que generalmente dispone de memoria y capacidad de procesamiento. (Impresoras, PCs, ...)
2. **Elementos de comportamiento:** Partes dinámicas de los modelos. Son dos:

- a) **Interacciones:** una interacción es un comportamiento que comprende un conjunto de mensajes intercambiados entre un conjunto de objetos, dentro de un contexto particular para alcanzar un propósito específico. El comportamiento de una sociedad de objetos o una operación individual puede especificarse mediante una interacción.
 - b) **Máquinas de estados:** una máquina de estados especifica las secuencias de estados por las que pasa un objeto o una interacción durante su vida en respuesta a eventos.
3. **Elementos de agrupación:** Son las partes organizativas de los modelos de UML. Solo existe una: el paquete, que es que es un mecanismo para organizar los elementos en grupos.
4. **Elementos de anotación:** Son comentarios que se pueden aplicar para describir, clarificar y hacer observaciones sobre cualquier elemento de un modelo. El principal elemento de anotación es la nota, que es simplemente un símbolo para mostrar restricciones y comentarios junto a un elemento o una colección de elementos.

Las **relaciones** entre los elementos estructurales son de cuatro tipos:

- **Dependencia:** es una relación semántica entre dos elementos, en la cual un cambio a un elemento (el elemento independiente) puede afectar a la semántica del otro elemento (el elemento dependiente).
- **Asociación:** Una **asociación** es una relación estructural que describe un conjunto de enlaces, los cuales son conexiones entre objetos. La **agregación** es un tipo especial de asociación, que representa una relación estructural entre un todo y sus partes. La **composición** es un tipo de agregación en el que cada parte solo puede pertenecer a un todo y no puede existir la parte sin el todo.
- **Generalización:** Una generalización es una relación de especialización /generalización en la cual los objetos del elemento especializado (el hijo) pueden sustituir a los objetos del elemento

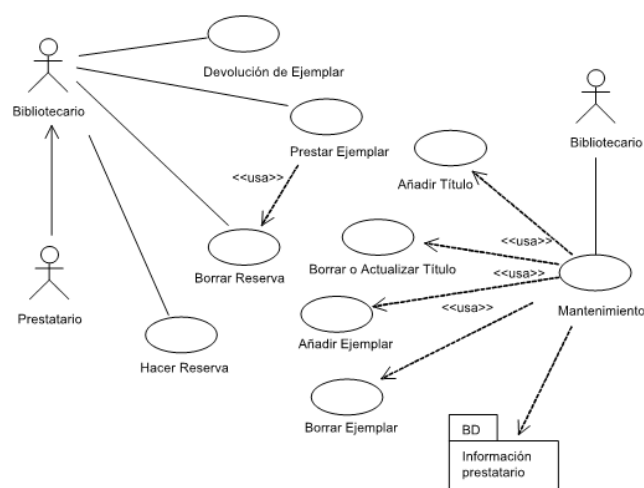
general (el padre). De esta forma, el hijo comparte la estructura y el comportamiento del padre.

- **Realización:** Es una relación semántica entre elementos, en donde un elemento especifica un contrato que otro elemento garantiza que cumplirá. Pueden ser entre interfaces y las clases y componentes que las realizan, y entre los casos de uso y las colaboraciones que los realizan.

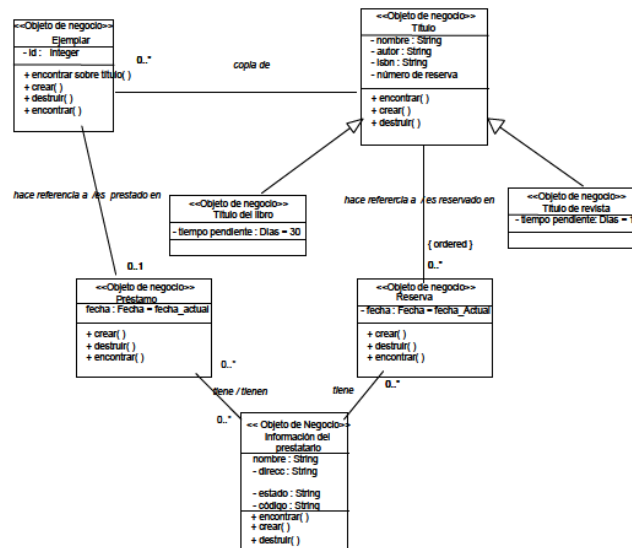
Por último, los **diagramas** son la representación gráfica de un conjunto de elementos, en general visualizado como un grafo conexo de nodos (elementos) y arcos (relaciones). Los diagramas se dibujan para visualizar un sistema desde diferentes perspectivas. Se pueden agrupar en dos bloques en función de si vemos el modelo de forma estática (estructural) o de forma dinámica (comportamiento). La primera incluye los diagramas de despliegue, componentes, clases y objetos, mientras que la segunda incluye los diagramas de estados, actividades, secuencia, colaboración y casos de uso.

1. **Diagrama de casos de uso.** Un caso de uso es una secuencia de acciones realizadas por el sistema, que producen un resultado observable y valioso para un usuario en particular, es decir, representa el comportamiento del sistema con el fin de dar respuestas a los usuarios. Los escenarios son los distintos caminos por los que puede evolucionar un caso de uso, dependiendo de las condiciones que se van dando en su realización. Los diagramas están formados por dos elementos: **actores**, que es algo o alguien que se encuentra fuera del sistema y que interactúa con él, y **casos de uso**, que representa el comportamiento que ofrece el sistema de información desde el punto de vista del usuario. Entre estos elementos se pueden dar tres tipos de relaciones: comunica (es la relación entre un actor y un caso de uso, que denota la participación del actor en dicho caso de uso), usa (relación de dependencia entre dos casos de uso que denota la inclusión del comportamiento de un

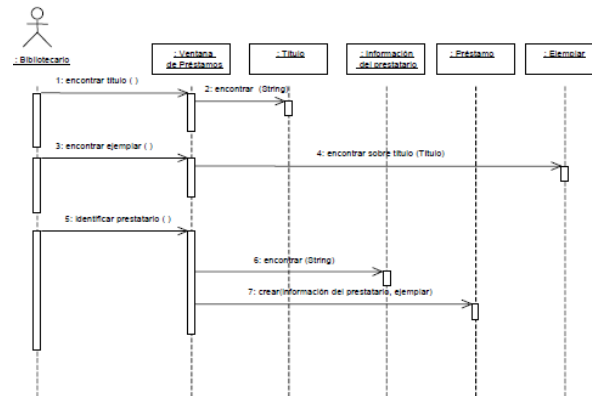
escenario en otro. Se usa cuando se quiere reflejar un comportamiento común en varios casos de uso) y extiende (relación de dependencia entre dos casos de uso en el que uno es una especialización del otro).



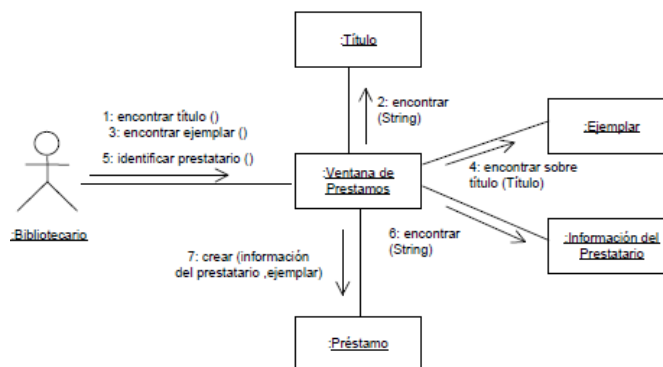
2. **Diagrama de clases:** Representa los aspectos estáticos del sistema, utilizando diversos mecanismos de abstracción (clasificación, generalización, agregación). Recoge las clases de objetos y sus asociaciones. En este diagrama se representa la estructura y el comportamiento de cada uno de los objetos del sistema y sus relaciones con los demás objetos. El modelo está formado por: **clases** (se representa como una caja dividida en tres zonas. En la zona superior se pone el nombre de la clase, en el centro se colocan los atributos (características) de la clase y en la zona inferior se incluye una lista con las operaciones que proporciona la clase), **relaciones** (asociaciones, agregaciones, composiciones, dependencias o herencia), **interfaces y paquetes**.



3. **Diagrama de objetos:** Modelan las instancias de elementos contenidos en los diagramas de clases. Muestra un conjunto de objetos y sus relaciones en un momento concreto. La representación gráfica de un objeto en UML es igual que la de una clase pero con el nombre subrayado.
4. **Diagramas de Interacción.** Se utilizan para modelar los aspectos dinámicos del sistema. Existen dos tipos de diagramas de interacción: **secuencia** y **colaboración**. Ambos son equivalentes. La diferencia entre ellos está en los aspectos que resaltan. Los diagramas de secuencia destacan el orden temporal de los mensajes, mientras que los diagramas de colaboración destacan la organización estructural de los objetos.
 - a. **Diagramas de secuencia:** Muestran las interacciones entre objetos ordenadas en **secuencia temporal**. Muestra los objetos que se encuentran en el escenario y la secuencia de mensajes intercambiados entre ellos para llevar a cabo la funcionalidad descrita por el escenario.

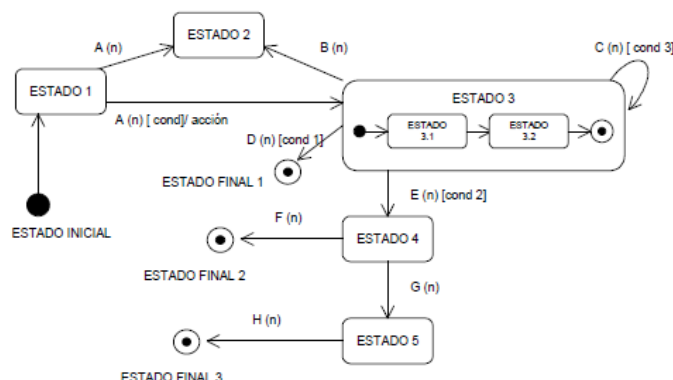


- b. **Diagramas de colaboración:** Es una forma alternativa al diagrama de secuencia para mostrar un escenario. Muestra la misma información pero de forma diferente. En los diagramas de colaboración no existe una secuencia temporal. Este diagrama resalta la **organización estructural** de los objetos que envían y reciben los mensajes. Este tipo de diagrama muestra un conjunto de objetos, enlaces entre ellos y los mensajes que intercambian.

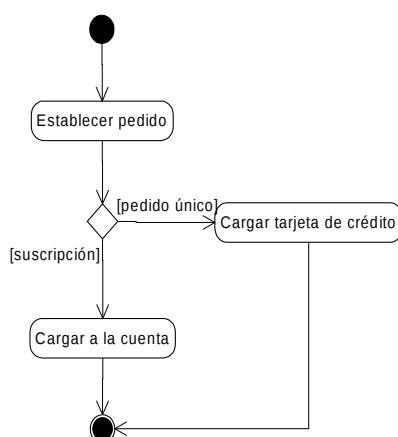


5. **Diagramas de estados:** Representan los estados que puede tomar un componente o un sistema y muestra los eventos que implican el cambio de un estado a otro. Los dos elementos principales en estos diagramas son los **estados** (representa algún comportamiento que es observable externamente y que perdura durante un periodo de tiempo finito. Viene dado por el valor de uno o varios atributos que lo caracterizan en un momento dado) y las posibles **transiciones** entre ellos. Una transición es un cambio de estado producido por un evento

y refleja los posibles caminos para llegar a un estado final desde un estado inicial. Un sistema sólo puede tener un **estado inicial**, que representa el punto de partida. El **estado final** representa que un componente ha dejado de tener cualquier interacción o actividad. Puede haber varios estados finales en un diagrama.



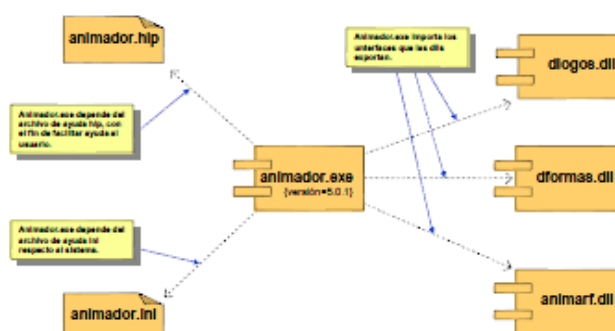
6. **Diagrama de actividades.** Puede considerarse un caso especial del diagrama de estados en el cual casi todos los estados son estados acción (identifican una acción que se ejecuta al entrar en él) y casi todas las transiciones evolucionan al término de dicha acción. Se suelen utilizar para modelar los pasos de un algoritmo.



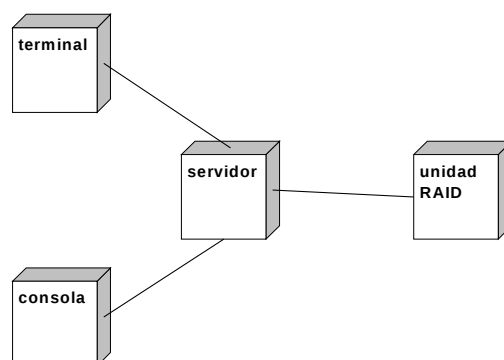
7. **Diagramas de implementación.** Un diagrama de implementación muestra las dependencias entre las partes de código del sistema (diagrama de componentes) o la estructura del sistema en ejecución (diagrama de despliegue). Los diagramas de componentes se utilizan para modelar la vista de implementación estática de un sistema,

mientras que los diagramas de despliegue se utilizan para modelar la vista de despliegue estática.

8. **Diagrama de componentes:** Muestra las organizaciones y dependencias lógicas entre componentes software, sean éstos componentes de código fuente, binarios, documentos, archivos o ejecutables. Los diagramas de componentes pueden contener paquetes para organizar los elementos.



9. **Diagrama de despliegue.** Muestra las relaciones físicas entre los componentes hardware y software en el sistema final, es decir, la configuración de los elementos de procesamiento en tiempo de ejecución y los componentes software (proceso y objetos que se ejecutan en ellos). Cubre principalmente la distribución, entrega e instalación de las partes que configuran el sistema físico, y se suelen utilizar para modelar sistemas empotrados, sistemas cliente-servidor y sistemas distribuidos.



32.2 Diseño de un sistema de información

Se define el **Diseño del Sistema** como *el proceso de definición de la arquitectura software: componentes, módulos, interfaces, procedimientos de prueba y datos de un sistema que se crean para satisfacer unos requisitos especificados y con los suficientes detalles como para permitir su implementación física*. Según la Metodología Métrica V3, el objetivo del proceso de Diseño del Sistema de Información es la definición de la arquitectura del sistema y del entorno tecnológico que le va a dar soporte, junto con la especificación detallada de los componentes del sistema de información.

32.2.1 Diseño estructurado.

El diseño estructurado se ocupa de la identificación, selección y organización de los módulos y sus relaciones. Se decide que componentes son necesarios y la interconexión entre los mismos para solucionar un problema bien especificado. Se comienza con la especificación resultante del proceso de análisis y se realiza una descomposición del sistema en módulos estructurados en jerarquías de tal modo que permitan la implementación de un sistema que no requiera elevados costes de mantenimiento.

Los principios que se consideran generalmente aceptados para las técnicas de diseño estructurado son los siguientes:

- **Modularización.** Se persigue que la Arquitectura técnica del sistema se fundamente en módulos de pequeño tamaño.
- **Independencia modular.** Cada uno de los módulos del sistema debe orientarse a que realice una función bien definida e independiente, lo cual permita “aislar” el origen de un problema o el área del sistema a modificar en caso preciso.
- **Modelización conceptual.** La estructuración del sistema en módulos no debe atender a un criterio exclusivamente técnico en sentido reducido, sino que debe reflejar conceptualmente la

estructura de la organización a la que sirve, de modo que se facilite su comprensión.

- **Principio de “caja negra”**. Una correcta definición de las interfaces permitirá ignorar la estructura interna de los módulos (es decir, considerarlos como una “caja negra”) y centrar la atención exclusivamente en los resultados que devuelve.

La técnica más importante dentro del Diseño Estructurado es el Diagrama de Estructuras (Structure chart), también denominada Diagrama de Estructura de Cuadros. Los **diagramas de estructura** son una herramienta para modelar los módulos de un sistema y sus relaciones y, junto con las especificaciones de funcionalidad de los módulos y las estructuras de datos, componen un diseño inicial que deberá ser analizado y mejorado. Permiten modelar un programa como una jerarquía de módulos. Cada nivel de la jerarquía representa una descomposición más detallada del módulo del nivel superior. La notación usada se compone básicamente de tres símbolos:

- **Módulos**: un conjunto de instrucciones que ejecutan alguna actividad y se presenta como una caja negra. Desde un punto de vista práctico, un módulo es una colección de instrucciones de un programa con cuatro características básicas: *Entradas y Salidas*, es decir, lo que un módulo recibe en una invocación y que necesita y lo que retorna como resultado; *Función* que son las actividades que un módulo hace con la entrada para producir la salida; la *Lógica Interna* por la cual se ejecuta la función; y *Estado Interno* que es su área de datos privada, datos para los cuales sólo el módulo hace referencia. Las entradas y salidas y las funciones proveen una visión externa del módulo, mientras que la lógica interna representa la visión interna del módulo. Un módulo se presenta como una caja, con un nombre en el interior que representa su función y las entradas y salidas representadas por pequeñas flechas que entran y salen del módulo.

- **Invocaciones:** los diagramas de estructura muestran las invocaciones que un módulo hace a otros módulos. Estas invocaciones son diseñadas como una flecha que sale del módulo llamador y apunta al módulo llamado. Los diagramas de estructura no tienen especificado el orden de invocación de los módulos invocados.
- **Cuplas:** Son las comunicaciones entre módulos. Existen varios tipos, basados en lo que ellas pueden producir en el módulo receptor:
 - o Cupla de Datos. Transporta datos a un módulo. Son también conocidos como parámetros de entrada del módulo.
 - o Cupla Modificada. Con una flecha doble (apuntando al modulo llamador y al módulo llamado) se especifica un argumento enviado a un módulo que deberá modificar su valor y del volver este valor modificado. Parámetros de entrada/salida
 - o Cupla de Resultados. Existen módulos que retornan valores sin la necesidad de que estén inicializados en el momento que se invocan.

Como dijimos al principio al hablar de los diagramas de estructuras, estos constituyen una aproximación inicial, que debe ser analizada y mejorada. Entre los criterios que se utilizan para mejorar el diseño tenemos:

- **Acoplamiento:** El acoplamiento entre módulos clasifica el grado de independencia entre pares de módulos. El objetivo es minimizar el acoplamiento, es decir, maximizar la independencia entre módulos.
- **Cohesión:** es la medida de intensidad de asociación funcional de los elementos de un módulo. El objetivo del diseño estructurado es obtener módulos altamente cohesivos, cuyos elementos estén fuerte y genuinamente relacionados unos con otros. Por otro lado, los elementos de un módulo no deberían estar fuertemente relacionados con elementos de otros módulos, porque eso llevaría a un fuerte acoplamiento entre ellos.

- **Descomposición** (Factoring): La descomposición es la separación de una función contenida en un módulo, para un nuevo módulo. Puede ser hecha para reducir el tamaño del módulo, hacer el sistema más claro, minimizar la duplicación de código, separar el trabajo de la coordinación de trabajos, o crear módulos más generales y fácilmente reutilizables.
- **Fan-Out:** El fan-out de un módulo es usado como una medida de complejidad. Es el número de subordinados inmediatos de un módulo (cantidad de módulos invocados).
- **Fan-In:** El fan-in de un módulo es usado como una medida de reusabilidad. Es el número de superiores inmediatos de un módulo (la cantidad de módulos que lo invocan).

32.2.2 Diseño de datos

El diseño de datos (a veces llamado arquitectura de datos) crea un modelo de datos y/o información que se representa con un alto nivel de abstracción (la visión de datos del cliente/usuario). Este modelo de datos es entonces refinado en progresivas representaciones específicas de la implementación, que pueden ser procesadas por un sistema basado en computadora. La estructura de datos ha sido siempre una parte importante del diseño de software.

Los objetos de datos definidos durante el análisis de requisitos del software son modelados utilizando diagramas entidad-relación y el diccionario de datos. La actividad de diseño de datos traduce esos elementos del modelo de requisitos en estructuras de datos a nivel de los componentes del software y, cuando es necesario, a arquitectura de base de datos a nivel de aplicación.

El diseño de datos a nivel de componentes se centra en la representación de estructuras de datos a las que se accede directamente a través de uno o más componentes del software. En realidad, el diseño de datos empieza

durante la creación del modelo de análisis. Wasserman ha propuesto un conjunto de principios que pueden emplearse para especificar y diseñar dicha estructura de datos, que pueden integrarse en las fases de análisis y diseño:

- 1) Los principios del análisis sistemático aplicados a la función y al comportamiento deberían aplicarse también a los datos. Invertimos mucho tiempo y esfuerzo en obtener, revisar y especificar los requisitos funcionales y el diseño preliminar. Las representaciones de flujo de datos y contenido deberían desarrollarse y revisarse, las de objetos de datos deberían identificarse, se deberían estudiar las organizaciones alternativas de datos y debería evaluarse el impacto del modelado de datos sobre el diseño del software. Una organización de datos alternativa nos podría llevar a obtener mejores resultados.
- 2) Todas las estructuras de datos y las operaciones a llevar a cabo en cada una de ellas deberían estar claramente identificadas. El diseño de una estructura de datos eficaz debe tener en cuenta las operaciones que se van a llevar a cabo sobre dicha estructura de datos. La especificación de los tipos de datos abstractos puede simplificar considerablemente el diseño del software.
- 3) Se debería establecer un diccionario de datos y usarlo para definir el diseño de los datos y del programa. Un diccionario de datos representa explícitamente las relaciones entre los objetos de datos y las restricciones de los elementos de una estructura de datos. Los algoritmos que deben aprovecharse de estas relaciones específicas pueden definirse más fácilmente si existe una especificación de datos tipo diccionario.
- 4) Las decisiones de diseño de datos de bajo nivel deberían dejarse para el final del proceso de diseño. Se puede usar un proceso de refinamiento paso a paso para el diseño de datos. Es decir, la organización general de datos puede definirse durante el análisis de requisitos; refinarse durante los trabajos de diseño de datos y especificarse en detalle durante el diseño a nivel de componentes. El

enfoque descendente del diseño de datos proporciona ventajas análogas al enfoque descendente del diseño de software: se diseñan y evalúan primeramente los atributos estructurales principales de manera que se pueda establecer la arquitectura de los datos.

- 5) La representación de las estructuras de datos deberían conocerla sólo aquellos módulos que deban hacer uso directo de los datos contenidos dentro de la estructura. El concepto de ocultación de información y el concepto relacionado de acoplamiento proporcionan una importante visión de la calidad del diseño del software. Además, se debe separar la visión lógica de un objeto de datos de su visión física.
- 6) Debería desarrollarse una biblioteca de estructuras de datos útiles y de las operaciones que se les pueden aplicar. Las estructuras de datos y las operaciones deberían considerarse como recursos para el diseño del software. Las estructuras de datos pueden diseñarse para que se puedan reutilizar. Una biblioteca de plantillas de estructuras de datos (tipos abstractos de datos) puede reducir el esfuerzo del diseño y de la especificación de datos.
- 7) Un diseño del software y un lenguaje de programación debería soportar la especificación y realización de los tipos abstractos de datos. La implementación de una estructura de datos sofisticada puede hacerse excesivamente difícil si no existen los medios de especificación directa de la estructura en el lenguaje de programación escogido para dicha implementación.

32.2.3 Diseño de la interfaz de usuario.

Todo equipo, y dentro de éste toda aplicación software, necesita poder comunicarse con el usuario de alguna forma. La idea fundamental en el concepto de interfaz es la de mediación entre hombre y máquina. Podemos definir entonces la Interfaz de Usuario (en adelante IU), como el conjunto de elementos hardware y software de un equipo que presentan información

al usuario y le permiten interactuar con la información y con el equipo. Es decir, es la cara que va a presentar el equipo (y dentro de ésta cada aplicación software) al usuario para poder comunicarse con él.

Dentro de la interfaz de usuario tendremos dos elementos:

- **Parte Hardware.** Es lo referente a la parte física del ordenador, los dispositivos utilizados para introducir, procesar y entregar los datos, básicamente: teclado, ratón y monitor.
- **Parte Software.** Es en la que suele pensarse cuando se habla de Interfaz de Usuario. Es la relativa a presentación que muestra, normalmente a través del monitor, una aplicación software.

La evolución de las interfaces va en paralelo al sistema operativo, pudiendo tener los siguientes tipos de interfaces:

- **Interfaz de línea de comandos:** el usuario escribe órdenes usando un lenguaje formal con vocabulario y sintaxis propia. Se usa un teclado, típicamente, y las órdenes están encaminadas a realizar una acción y mostrar los resultados al usuario.
- **Interfaz a través de Menús:** Un Menú es una lista de opciones que se muestran en pantalla o en una ventana de la pantalla para que los usuarios elijan la opción que deseen. Tenemos distintos tipos: Menús de pantalla completa, menús de barra, en cascada, paleta de herramientas o menús contextuales.
- **Interfaces gráficos de usuario.** La idea básica o paradigma de las interfaces gráficas es: *Lo que ves es lo que puedes conseguir* (en inglés, la conocida frase: *What you see is what you get*, WYSIWYG) y esto se consigue fundamentalmente con la manipulación directa del Interfaz de usuario a través de iconos y gráficos en general. De hecho, la interfaz de usuario gráfica de un sistema junto con sus dispositivos de la entrada a veces es llamado: *look-and-feel* (mirar y sentir). Un GUI es una representación gráfica en la pantalla del ordenador de los programas, datos y objetos, así como de la interacción con ellos. Proporciona al usuario las herramientas para

realizar sus operaciones, más que una lista de las posibles operaciones. Aunque el uso de los GUI es más sencillo que el de las interfaces de línea de comandos, la introducción de instrucciones es más lenta, por lo que las GUI suelen tener la opción de emplear un sistema equivalente al de línea de instrucciones como alternativa rápida para los usuarios más expertos.

Existen tres puntos de vista distintos, también llamados modelos, en el diseño de IU:

- Modelo del **Usuario**: El usuario tiene su visión personal del sistema, y espera que éste se comporte de una cierta forma.
- Modelo del **Programador**: El programador no tiene que considerar la parte visual de interacción con el usuario, sino el funcionamiento interno de la aplicación. Los objetos que manipula el programador, son distintos de los que trata el usuario.
- Modelo del **Diseñador**: El diseñador debe tener un punto de vista que mezcle las necesidades, ideas y deseos del usuario con los materiales de que dispone el programador para diseñar un producto de software. Es un intermediario entre ambos. El modelo del diseñador describe los objetos que utiliza el usuario, su presentación al mismo y las técnicas de interacción para su manipulación. Consta de tres partes: **presentación**, **interacción** y **relaciones** entre los objetos.

Existen varios principios relevantes para el diseño e implementación de IU, sobre todo para los IU gráficas, siendo algunos de los más importantes son los siguientes

- **Anticipación**. Las aplicaciones deberían intentar anticiparse a las necesidades del usuario y no esperar a que el usuario tenga que buscar la información, recopilarla o invocar las herramientas que va a utilizar.

- **Autonomía.** El ordenador, la IU y el entorno de trabajo deben estar a disposición del usuario. Se debe dar al usuario el ambiente flexible para que pueda aprender rápidamente a usar la aplicación. Sin embargo, está comprobado que el entorno de trabajo debe tener ciertas cotas, es decir, ser explorable pero no azaroso.
- **Percepción del color.** Aunque se utilicen convenciones de color en la IU, se deberían usar otros mecanismos secundarios para proveer la información a aquellos usuarios con problemas en la visualización de colores
- **Valores por defecto.** Los valores por defecto deberían ser opciones inteligentes y sensatas. Además, los mismos tienen que ser fáciles de modificar.
- **Consistencia.** Para lograr una mayor consistencia en la IU se requiere profundizar en diferentes aspectos que están catalogados en niveles. Se realiza un ordenamiento de mayor a menor consistencia:
 - o **Interpretación del comportamiento del usuario:** La IU debe comprender el significado que le atribuye un usuario a cada requerimiento. Ejemplo: mantener el significado de los comandos abreviados (shortcutkeys) definidos por el usuario.
 - o **Estructuras invisibles:** se requiere una definición clara de las mismas, ya que sino el usuario nunca podría llegar a descubrir su uso. Ejemplo: ampliación de ventanas mediante la extensión de sus bordes.
 - o **Pequeñas estructuras visibles:** se puede establecer un conjunto de objetos visibles capaces de ser controlados por el usuario, que permitan ahorrar tiempo en la ejecución de tareas específicas. Ejemplo: icono y/o botón para impresión.
 - o **Una sola aplicación o servicio:** la IU permite visualizar a la aplicación o servicio utilizado como un componente único. Ejemplo: La IU despliega un único menú, pudiendo además acceder al mismo mediante comandos abreviados.

- o **Un conjunto de aplicaciones o servicios:** la IU visualiza a la aplicación o servicio utilizado como un conjunto de componentes. Ejemplo: La IU se presenta como un conjunto de barras de comandos desplegadas en diferentes lugares de la pantalla, pudiendo ser desactivadas en forma independiente.
- o **Consistencia del ambiente:** la IU se mantiene en concordancia con el ambiente de trabajo. Ejemplo: La IU utiliza objetos de control como menús, botones de comandos de manera análoga a otras IU que se usen en el ambiente de trabajo.
- o **Consistencia de la plataforma:** La IU es concordante con la plataforma. Ejemplo: La IU tiene un esquema basado en ventanas, el cual es acorde al manejo del sistema operativo utilizado.
- **Eficiencia del usuario.** Se debe considerar la productividad del usuario antes que la productividad de la máquina. Si el usuario debe esperar la respuesta del sistema por un período prolongado, estas pérdidas de tiempo se pueden convertir en pérdidas económicas para la organización. Los mensajes de ayuda deben ser sencillos y proveer respuestas a los problemas. Los menús y etiquetas de botones deberían tener las palabras claves del proceso.
- **Ley de Fitt.** Según dice la ley de Fitt: “El tiempo para alcanzar un objetivo es una función de la distancia y tamaño del objetivo”. Es por ello que es conveniente usar objetos grandes para las funciones importantes.
- **Interfaces explorables.** Siempre que sea posible se debe permitir que el usuario pueda salir ágilmente de la IU, dejando una marca del estado de avance de su trabajo, para que pueda continuarlo en otra oportunidad. Para aquellos usuarios que sean novatos en el uso de la aplicación, se deberá proveer de guías para realizar tareas que no sean habituales. Siempre se debe contar con un comando

“deshacer”. Este suprimirá la necesidad de tener que contar con diálogos de confirmación para cada acción que realice en sistema. El usuario debe sentirse seguro de poder salir del sistema cuando lo desee. La IU debe tener un objeto fácil de accionar con el cual poder finalizar la aplicación.

- **Uso de metáforas.** Las buenas metáforas crean figuras mentales fáciles de recordar. La IU puede contener objetos asociados al modelo conceptual en forma visual, con sonido u otra característica perceptible por el usuario que ayude a simplificar el uso del sistema.
- **Curva de aprendizaje.** El aprendizaje de un producto y su usabilidad no son mutuamente excluyentes. El ideal es que la curva de aprendizaje sea nula, y que el usuario principiante pueda alcanzar el dominio total de la aplicación sin esfuerzo.
- **Reducción de latencia.** Siempre que sea posible, el uso de tramas (multi-threading) permite colocar la latencia en segundo plano (background). Las técnicas de trabajo multitarea posibilitan el trabajo ininterrumpido del usuario, realizando las tareas de transmisión y computación de datos en segundo plano.
- **Protección del Trabajo.** Se debe poder asegurar que el usuario nunca pierda su trabajo, ya sea por error de su parte, problemas de transmisión de datos, de energía, o alguna otra razón inevitable.
- **Legibilidad.** Para que la IU favorezca la usabilidad del sistema de software, la información que se exhiba en ella debe ser fácil de ubicar y leer. Para lograr obtener este resultado se deben tener en cuenta algunas premisas como: el texto que aparezca en la IU debería tener un alto contraste, se debe utilizar combinaciones de colores como el texto en negro sobre fondo blanco o amarillo suave. El tamaño de las fuentes tiene que ser lo suficientemente grande como para poder ser leído en monitores estándar. Es importante hacer clara la presentación visual (colocación/agrupación de objetos, evitar la presentación de excesiva información).

32.2.4 Diseño procedimental.

El **diseño procedimental** se realiza después de que se ha establecido la estructura del programa y de los datos. Transforma los elementos estructurales en una descripción de los elementos del software. Define los algoritmos de procesamiento necesarios. Debe especificar los detalles de los procedimientos sin ambigüedad y además debe facilitar la codificación, de forma que el código se obtenga de forma natural a partir del diseño.

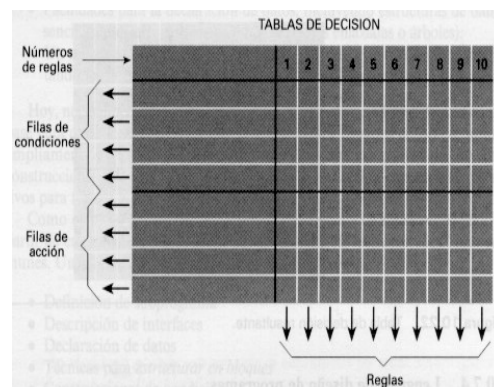
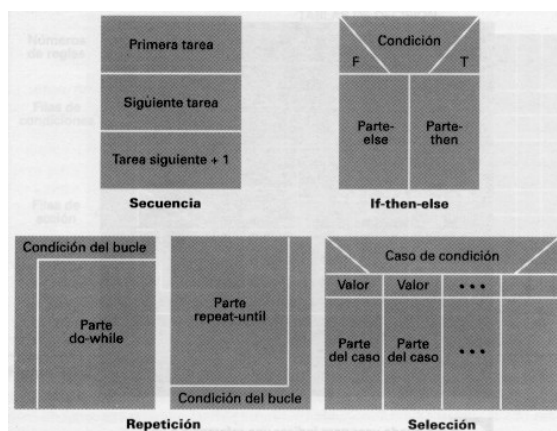
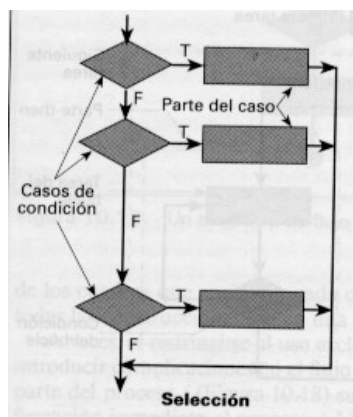
A finales de los 60 se propuso la utilización de un conjunto de construcciones lógicas con las que podía formarse cualquier programa. Estas construcciones serían la **secuencia**, la **condición** y la **repetición**, y cualquier programa podría construirse utilizando solo este tipo de construcciones. Estas son uno de los fundamentos de la programación estructurada. Las construcciones estructuradas se propusieron para limitar el diseño procedimental del software a un número pequeño de operaciones predecibles, lo que facilita la legibilidad, prueba y mantenimiento del software.

Para la especificación procedimental que define los algoritmos se podría utilizar el lenguaje natural, pero debido a la cantidad de ambigüedades que este lenguaje acarrea, es necesario utilizar una forma más restringida de representación. Y entre estas formas de representación tenemos:

- **Diagrama de flujo:** Es la representación gráfica que más se utiliza en el diseño procedimental. Para representar un paso de procesamiento se utiliza un cuadro, para representar una condición se utiliza un rombo, y para representar el flujo de control se utilizan flechas.
- **Diagrama de cajas (N-S):** Esta notación surgió del deseo de desarrollar una representación para el diseño procedimental que no permitiera la violación de construcciones estructuradas. Estos

diagramas fueron desarrollados por Nassi y Schneiderman y perfeccionados por Chapin, y tienen las características siguientes:

- o El ámbito funcional está bien definido y es claramente visible.
 - o La transferencia de control arbitraria es imposible.
 - o Es fácil determinar el ámbito de los datos locales y globales.
 - o La recursividad es fácil de representar
- **Tablas de decisión.** Constituyen una notación que traduce las acciones y condiciones a una forma tabular. Los pasos para la creación de una tabla de decisión son los siguientes:
 - o Listar todas las acciones que se pueden asociar a un módulo.
 - o Listar todas las condiciones necesarias para la ejecución de un procedimiento.
 - o Asociar conjuntos de condiciones específicas a acciones específicas, eliminando combinaciones imposibles. Desarrollar las posibles combinaciones de condiciones.
 - o Definir reglas indicando las acciones que ocurren para una serie de condiciones.
 - **Lenguaje de diseño de programas.** Un lenguaje de diseño de programas (LDP), también conocido como lenguaje estructurado o pseudocódigo es un lenguaje que utiliza el vocabulario de un lenguaje y la sintaxis de otro. Independientemente de su origen, un LDP tiene que tener las características siguientes:
 - o Una sintaxis fija de palabras clave que permitan construir todas las construcciones estructuradas, declarar datos y establecer características de modularidad.
 - o Una sintaxis libre en lenguaje natural para describir las características de procesamiento.
 - o Facilidades para la declaración de datos, incluyendo estructuras de datos simples y complejas.
 - o Un mecanismo de definición de subprogramas y de llamada a éstos.



32.2.5

Diseño

orientado a objetos.

El diseño orientado a objetos transforma el modelo de análisis creado usando análisis orientado a objetos, en un modelo de diseño que sirve para la construcción de software que podrá ser implementado en un lenguaje de programación. Se deben identificar los objetos pertinentes, clasificarlos dentro de las clases en la granularidad correcta, definir interfaces de clases y jerarquías de herencia y establecer relaciones clave entre ellos, de tal forma que se obtenga la solución al problema del usuario. El diseño debe ser específico al problema que se tiene entre manos, pero suficientemente general para adaptarse a problemas y requerimientos futuros.

Los métodos estructurados y sus correspondientes notaciones definen un sistema como un conjunto secuencial de módulos interdependientes que comparten datos. En cambio, los métodos orientados a objetos definen un conjunto de módulos independientes relacionados y con visibilidad limitada entre ellos.

Esencialmente, el DOO consta de los tres siguientes pasos:

- **Identificación y definición de objetos y clases.** El principal problema del desarrollo de un sistema orientado a objetos es encontrar los objetos en la fase de AOO y DOO. Un posible método para encontrarlos es el método gramatical propuesto por Booch que

analiza la descripción textual del sistema y propone a los sustantivos como posibles identificadores de las clases y los verbos como posibles métodos de las clases. La lista resultante de clases (nombres) y métodos (verbos) se utilizará para comenzar el proceso de diseño. Una vez identificados los objetos, se detallan los atributos de cada uno. Con el **método de Booch** para encontrar clases es difícil conseguir un resultado de alta calidad, pues la abstracción de clases que consigamos, depende de una estructuración inteligente de la descripción del problema en elementos independientes e intuitivamente correctos. Otro modelo que utiliza el método gramatical para la identificación de clases y objetos es el **modelo CRC** (Clases - Responsabilidades - Colaboraciones). Un modelo CRC es una colección de tarjetas que representan clases. Las tarjetas están divididas en tres secciones. A lo largo de la cabecera de la tarjeta se escribe el nombre de la clase. En el cuerpo se listan las responsabilidades de la clase a la izquierda y a la derecha los colaboradores. Las **responsabilidades** estarían formadas por los atributos y operaciones de las clases. Las clases cumplen con sus responsabilidades de dos formas: O bien una clase puede usar sus propias operaciones para manipular sus propios atributos, cumpliendo por lo tanto con una responsabilidad particular, o puede colaborar con otras clases.

Se han definido una serie de directrices que facilitan la identificación y definición de clases y métodos:

- o Modelar con clases las entidades que ocurren de forma natural en el problema.
- o Diseñar métodos con una sola finalidad.
- o Diseñar un método nuevo antes de ampliar uno existente.
- o Evitar métodos extensos.
- o Guardar como instancia los datos necesitados por más de un método o por una subclase.

- o El diseñador debe trabajar para obtener una biblioteca de clases, y no para él mismo, ni para desarrollar el sistema actual.

Además, para evitar la creación de clases innecesarias o declaración de clases que no lo sean, una clase debe ofrecer una serie de servicios a objetos de un tipo determinado. Una clase se debería crear cuando:

- o La nueva clase represente una abstracción significativa del problema
 - o Es probable que los servicios que proporciona sean utilizados por otras clases.
 - o Su comportamiento sea complejo.
 - o Si se representara como un método de otra clase, pocos usuarios de ésta lo invocarían.
- **Organización de relaciones entre clases.** Una vez identificadas las clases deberíamos encontrar las relaciones entre estas. Para ello podríamos usar distintas alternativas, como el modelo Clases-Responsabilidades-Colaboraciones (CRC) visto anteriormente. Cada tarjeta del modelo CRC contiene una clase con una lista de responsabilidades. El siguiente paso es definir aquellas clases colaboradoras que ayudan en la realización de cada responsabilidad. Esto establece las **conexiones o relaciones** entre clases. Las **relaciones** pueden derivarse a partir del examen de los verbos en la descripción del problema. Una vez conectadas las clases con sus colaboradores, etiquetamos cada una de estas conexiones, les añadimos una dirección, en función de qué clase llama a qué otra y por último se evalúa cada extremo de la conexión para determinar la cardinalidad. Este modelo de clases conectadas da lugar al **modelo Objeto-Relación**.
 - **Extracción de estructuras en una jerarquía de clases.** Una vez definidos los objetos y sus relaciones, el paso siguiente consiste en

observar características comunes para crear abstracciones a nivel de clase, pero no existe ninguna metodología formal para la realización de estas abstracciones. Uno de los objetos de un diseño orientado a objetos es guardar propiedades y métodos de clases en el nivel de abstracción más alto posible, de forma que los compartan la mayoría de clases y se fomente la reutilización. Para este paso se emplean los diagramas de generalización o herencia.

Aunque desde los inicios del desarrollo de software orientado a objetos han existido múltiples metodologías, como por ejemplo Object-Oriented Design (OOD, Booch), Object-Modeling Technique (OMT, Rumbaugh), Object Oriented Analysis (OOA, Coad/Yourdon), Hierarchical Object Oriented Design (HOOD, ESA), Object Oriented Structured Design (OOSD, Wasserman), etc, el Lenguaje Unificado de modelado (UML) puso fin a la guerra de metodologías. Actualmente en la industria del desarrollo de software tenemos al UML como un estándar para el modelado de sistemas orientados a objetos que es usado tanto para la fase de análisis como desarrollo.

Durante el proceso de DOO, un ingeniero del software debe observar cada oportunidad en la que pueda reutilizar patrones de diseño existentes (cuando cumplen las necesidades del diseño), en vez de crear otros nuevos. Los patrones de diseño son la base para la búsqueda de soluciones a problemas comunes en el desarrollo de software y otros ámbitos referentes al diseño de interacción o interfaces. Un **patrón de diseño** es una solución a un problema de diseño. Para que una solución sea considerada un patrón debe poseer ciertas características. Una de ellas es que debe haber comprobado su efectividad resolviendo problemas similares en ocasiones anteriores. Otra es que debe ser reutilizable, lo que significa que es aplicable a diferentes problemas de diseño en distintas circunstancias.

Bibliografía

- Herramientas y Metodologías de Análisis y Diseño Estructurado. Apunte de la Cátedra Metodologías de Desarrollo de Software I. Claudia Marcos – Edgardo Belloni
- Ingeniería del Software. Un enfoque práctico. ROGER S. PRESSMAN. Ed. McGraw Hill
- Aprendiendo UML en 24 horas. Joseph Schmuller
- Métrica 3 – Técnicas y Prácticas. Ministerio de Administraciones Públicas
- Curso de OO dirigido por la introducción a la ambigüedad. Anexo 1 : UML
http://is.ls.fi.upm.es/docencia/proyecto/docs/curso/12Anexo_1_UML.doc
- Introducción a la programación I. Diagramas de estructuras. <http://www.exa.unicen.edu.ar/catedras/prog1/introprog1/sites/default/files/ApuntesDiagramaEstructura.pdf>
- Fundamentos de diseño del software y Diseño orientado a los objetos Asignatura Laboratorio de proyectos. Univ. de Almería. <http://indalog.ual.es/mtorres/LP/FundamentosDiseno.pdf>
<http://indalog.ual.es/mtorres/LP/DOO.pdf>
- http://es.wikipedia.org/wiki/Patrón_de_diseño

Autor: Hernán Vila Pérez

Jefe del Servicio de Informática. Instituto Galego de Vivenda e Solo

Vicepresidente del CPEIG